

ORBITGUARD

Autonomous Multi-Agent Conjunction Response System

PROJECT BIBLE — Bit N Build 2026 (Internal Round)

24-Hour Build · 4-Person Team · Single Source of Truth

Role	Owns
1. Architecture / Novelty / Presentation / Integration	This document, API contracts, pitch, cross-team integration
2. Backend	FastAPI service, SGP4 propagation, conjunction detection, maneuver + counterfactual simulation engine
3. Frontend	React + 3D globe dashboard, decision-trace UI, live data binding
4. AI Agent	Multi-agent orchestrator: Planner, Conflict-Check glue, Mission Guardian, Commander

Read your section fully before writing code. Section 4 (API Contracts) is binding for everyone — do not deviate from field names or types without updating this document and telling the group.

0. How To Use This Document

This is the single reference for the entire 24 hours. Every member reads Sections 1–4 (shared context and contracts) in the first hour, then jumps to their own role section. The API contracts in Section 4 are frozen after hour 1 — if a field must change, whoever needs the change pings the group immediately, not two hours later.

- **Section 1** — The idea, why it wins on the judging rubric, and the one sentence to say when a judge asks 'so what's new here?'
- **Section 2** — Full system architecture and tech stack.
- **Section 3** — 24-hour execution timeline (shared).
- **Section 4** — API and data contracts. Everyone's code must match this exactly.
- **Section 5** — Backend spec (Member 2).
- **Section 6** — AI Agent spec (Member 4).
- **Section 7** — Frontend + dashboard spec (Member 3).
- **Section 8** — Integration, Git workflow, and demo script (Member 1).
- **Section 9** — Fallback plans if something breaks with little time left.

1. The Idea, Locked (Final Scope)

One-line pitch: OrbitGuard is a Gemini-powered multi-agent conjunction response system that detects dangerous orbital encounters, generates and counterfactually simulates avoidance maneuvers, rejects plans that create secondary conflicts, and selects the safest feasible response for human approval.

The core question to open the pitch with: "What if the maneuver that saves a satellite from one collision creates another collision?" OrbitGuard answers this by simulating the consequences of its own proposed actions before accepting them. Say this in the first 15 seconds of the demo — everything else is supporting evidence.

1.1 Why this is innovative — three points, use these exact framings

- **Counterfactual self-checking.** The system does not blindly trust its first solution — it simulates the consequences of a proposed maneuver before accepting it.
- **Second-order conflict awareness.** A maneuver that solves one collision can create another. OrbitGuard explicitly searches for this consequence and rejects unsafe plans.
- **Agentic decision workflow.** The system moves through Observe → Reason → Use Tools → Simulate → Evaluate → Reject → Replan → Decide → Verify, rather than simply producing a single AI prediction.

1.2 Why this maps to the rubric

Judging Criterion	Where OrbitGuard scores
Problem Understanding & Impact	Space debris and conjunction events are a real, growing operational problem (Kessler-syndrome risk, rising satellite counts). Framed explicitly as a decision-support tool for a human operator, not a replacement for one — avoids overclaiming.
Innovation & Creativity	The novel part is not detection (that's a solved SGP4 problem) — it's the counterfactual self-check loop: propose → simulate consequences → reject → replan → verify. Most hackathon 'AI safety' demos never show the AI rejecting its own idea. This one visibly does, on stage, more than once.
Agentic AI Implementation	A real multi-step pipeline with distinct responsibilities, Gemini reasoning over tool results, memory of prior rejected plans, a final synthesis step that weighs competing objectives, and a closing verification pass before handoff to a human.
Technical Implementation	Real orbital mechanics (SGP4), a working simulation loop, a FastAPI backend, and a 3D live dashboard — not a mockup with fake numbers.

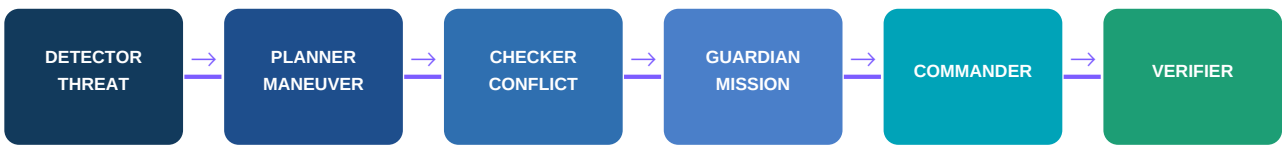
Judging Criterion	Where OrbitGuard scores
Solution Effectiveness & Usability	Output is a single, justified recommendation plus every rejected alternative and why — exactly the artifact a satellite operator needs to approve a fast decision.
Demo & Presentation	The 3-panel dashboard is built specifically to narrate itself: alert → plan tried → rejected → plan tried again → selected → verified → approved, visible as it happens.

1.3 Guardrails for the pitch (say these explicitly)

- "OrbitGuard is a decision-support prototype for autonomous conjunction response. It does not directly control real spacecraft." Say this line, close to verbatim, at least once.
- Orbital data is a controlled synthetic scenario for the demo, not live classified conjunction data — label it on screen as "Controlled simulation scenario."
- A human operator approves the final maneuver — the system recommends and verifies, a person clicks Approve.

2. System Architecture

2.1 High-level pipeline



Threat Detector finds the conjunction → Maneuver Planner proposes 3–4 candidate actions → Conflict Checker re-simulates the whole scene for each candidate → Mission Guardian scores candidates by satellite importance → Commander fuses everything into one decision → Verifier re-checks the selected maneuver once more before it goes to the human for approval.

2.2 Full stack

Layer	Technology	Notes
Frontend	React (Vite) + CesiumJS (or Three.js fallback, see 9.2) + Tailwind CSS	3D globe, 3-panel HUD dashboard, Approve Maneuver button
Realtime/data	REST polling (2–3s) or WebSocket (FastAPI native)	WebSocket preferred once basic REST works — do not build it first
Backend API	FastAPI (Python 3.11)	Single service, typed with Pydantic models matching Section 4
Orbital mechanics	sgp4 (PyPI package) + numpy	Propagates TLE-like orbital elements to position/velocity vectors
Conjunction & maneuver sim	Custom Python module using numpy/scipy	Pairwise distance search over a propagation window
AI reasoning	Python orchestrator classes + Google Gemini API (structured JSON output, function-calling-shaped) for reasoning/explanation/justification	Deterministic physics stays in Python; Gemini proposes/explains, it never invents numbers
State/storage	In-memory Python objects + JSON scenario file	No database needed for a 24h demo — do not add Postgres, it costs time and buys nothing here
Deployment	Everything runs locally (localhost) for the demo	Docker optional only if time allows; not required for judging

2.3 Why an LLM at all (and where it must NOT be used)

This is the detail most teams get wrong and it directly affects the 'Agentic AI' score. The physics (propagation, miss distance, delta-v, risk score, safety status) must be deterministic and correct — Gemini must never compute, invent, or override a physical value or the final numeric decision. Gemini's job is reasoning over already-computed structured results: interpreting threat data, comparing candidates, explaining why a plan was rejected in plain language, and writing the final justification. This keeps the demo reliable (no hallucinated physics) while still being genuinely agentic.

Agent	Deterministic Python	Gemini's role
Threat Detector	100% — propagation + distance search + risk formula	None

Agent	Deterministic Python	Gemini's role
Maneuver Planner	Computes the physical result of each candidate type	Optionally comments on which candidate types make sense given context — cuttable if short on time
Conflict Checker	100% — re-runs propagation for every candidate against every other object	None
Mission Guardian	Holds/looks up criticality weights	Explains in one sentence why criticality changed the ranking
Commander	Computes the final weighted score (formula in 6.4)	Writes the human-readable justification and rejection reasons shown on screen
Verifier	Re-runs the Conflict Checker once on the selected plan — no new logic, same function	Optionally narrates the verification result ("Final risk: LOW")

2.4 Two simplifications, locked — do not relitigate these mid-hackathon

- **No real multi-turn Gemini function-calling loop.** Python runs every tool first (propagate, detect, simulate) and hands Gemini the finished, structured results in a single prompt asking it to reason over them. This gets you the 'tool-augmented reasoning' story honestly, without the debugging risk of a live request-tool / execute / return-result loop the night before a demo.
- **Verifier is not a new subsystem.** It is the existing Conflict Checker function, called one more time on the selected plan. If it fails, allow exactly **one** replan attempt (try the next-best candidate), then stop — never build an open-ended retry loop. The synthetic scenario (Section 4.1 / 9.4) is constructed so Plan 3 always verifies clean; the demo path never depends on a loop that could genuinely spin.

3. 24-Hour Execution Timeline

This is a shared clock. Everyone commits to it. Mock data unblocks Frontend and AI Agent from ever waiting on Backend.

Hour	Backend (M2)	AI Agent (M4)	Frontend (M3)	Architecture/Integration (M1)
0–1	Set up FastAPI skeleton, freeze Section 4 contracts	Set up orchestrator skeleton + mock inputs	Set up React/Vite, 3-panel layout shell	Lock scope, create repo, branch strategy, write mock_scenario.json
1–4	Implement SGP4 propagation + conjunction detector against mock_scenario.json	Build Planner + Commander prompts against mocked detector output	Build static dashboard with mock JSON, no globe yet	Write OpenAPI stub, unblock everyone with mock endpoints
4–8	Maneuver generation (raise/lower/phase/wait) + delta-v calc	Wire Planner to real detector output via API	Integrate CesiumJS/Three globe, plot static orbits	First integration checkpoint: backend + AI talking
8–12	Counterfactual simulation loop (the novel part) — re-check all objects per candidate	Conflict Checker glue + Mission Guardian criticality weighting	Live orbit animation + alert panel wired to /threats	Second integration checkpoint, fix contract drift
12–16	Expose /simulate and /decision endpoints, add WebSocket if time allows	Commander final scoring + justification text generation	Decision-trace panel wired to /decision, plan-rejected animation	Full end-to-end run #1, log every bug
16–20	Bug fixes, edge cases (no feasible plan, tie-break)	Prompt tuning for clarity, cut latency, cache repeated calls	Visual polish per Section 7.6 (make it look aerospace, not AI)	Full end-to-end run #2, start writing demo script
20–22	Freeze backend	Freeze agent logic	Freeze UI, only fix visual bugs	Rehearse demo, record screen + voiceover
22–24	Buffer / final commits	Buffer / final commits	Buffer / final commits	Push public repo, upload demo video, submit form

Commit to GitHub every 3–6 hours minimum (hackathon rule) — align commits with the checkpoints above.

4. API & Data Contracts — BINDING FOR ALL 4 MEMBERS

Freeze these field names and types in hour 1. Backend implements them exactly. Frontend and AI Agent code against them from a shared `mock_scenario.json` before real endpoints exist, so no one is ever blocked waiting on someone else.

4.1 Scenario object (input)

```
{
  "id": "SAT-03",
  "name": "CommSat-Alpha",
  "type": "satellite",          // "satellite" | "debris"
  "criticality": "critical",    // "critical" | "standard" | "low"
  "orbital_elements": {
    "semi_major_axis_km": 7050.0,
    "eccentricity": 0.001,
    "inclination_deg": 51.6,
    "raan_deg": 120.4,
    "arg_perigee_deg": 45.0,
    "mean_anomaly_deg": 10.2,
    "epoch": "2026-09-12T10:00:00Z"
  }
}
```

Use synthetic Keplerian elements per object for the controlled demo (generated once, saved to `mock_scenario.json`) rather than pulling real-world TLE catalogs — simpler, reproducible, and you fully control when the conjunction happens.

4.2 GET /api/threats — conjunction detection result

```
{
  "threats": [
    {
      "id": "THREAT-001",
      "primary": "SAT-03",
      "secondary": "DEB-17",
      "tca": "2026-09-12T14:32:10Z",
      "miss_distance_km": 0.42,
      "relative_velocity_km_s": 7.8,
      "risk_score": 0.87,          // 0.0 - 1.0
      "severity": "HIGH"          // LOW | MEDIUM | HIGH
    }
  ]
}
```

4.3 POST /api/plan — maneuver candidates for a threat

Request: { "threat_id": "THREAT-001" }

```
{
  "threat_id": "THREAT-001",
  "candidates": [
    { "id": "PLAN-1", "type": "raise_orbit", "delta_v_ms": 3.1, "delay_min": 0 },
    { "id": "PLAN-2", "type": "lower_orbit", "delta_v_ms": 2.6, "delay_min": 0 },
    { "id": "PLAN-3", "type": "phase_shift", "delta_v_ms": 2.8, "delay_min": 15 },
    { "id": "PLAN-4", "type": "wait", "delta_v_ms": 0.0, "delay_min": 0 }
  ]
}
```

4.4 POST /api/simulate — counterfactual check (the novel part)

Request: { "threat_id": "THREAT-001", "plan_id": "PLAN-1" }. Every field here is exactly what the frontend's "why was this rejected" card (Section 7.2) renders, so do not rename them once frozen.

```
{
  "plan_id": "PLAN-1",
  "primary_resolved": true,
  "secondary_conflicts": [
    { "object": "SAT-05", "miss_distance_km": 0.31, "risk_score": 0.63 }
  ],
  "closest_approach_km": 0.31,
  "feasible": false,
  "status": "REJECTED",          // "REJECTED" | "SELECTED"
  "reason": "Although this plan resolves the original threat, its simulated "
            "trajectory creates a new high-risk conjunction with SAT-05."
}
```

4.5 POST /api/decide — Commander + Verifier final output

Request: { "threat_id": "THREAT-001" } — backend runs the full pipeline server-side, including the Verifier's re-check on the selected plan, and returns:

```
{
  "threat_id": "THREAT-001",
  "selected_plan": "PLAN-3",
  "scores": {
    "safety": 0.95, "secondary_conflicts": 0, "delta_v_ms": 2.8,
    "mission_criticality_weight": 1.0, "total_score": 0.91
  },
  "rejected_plans": [
    { "plan_id": "PLAN-1", "status": "REJECTED", "closest_approach_km": 0.31,
      "reason": "Creates a new high-risk conjunction with SAT-05" },
    { "plan_id": "PLAN-2", "status": "REJECTED", "closest_approach_km": 1.40,
      "reason": "Insufficient miss-distance margin at TCA" }
  ],
  "justification": "Phase shift resolves the primary conjunction with zero secondary "
                  "conflicts at an acceptable fuel cost, and the target is a critical "
                  "asset so cost is secondary to certainty.",
  "verification": {
    "status": "SAFE",          // "SAFE" | "FAILED"
    "closest_approach_km": 15.4,
    "replans_used": 0          // 0 or 1 — see 2.4, never more than 1
  },
  "requires_human_approval": true
}
```

If verification.status is "FAILED", the backend has already used its one allowed replan (2.4) against the next-best candidate before responding — the frontend never sees or handles a failed verification directly, it only ever receives a final, verified plan or a clearly labeled "no feasible plan" response.

4.6 POST /api/approve — human-in-the-loop sign-off

Request: { "threat_id": "THREAT-001", "plan_id": "PLAN-3" }. This does not command a real spacecraft — it flips a status flag the frontend uses to trigger the before/after orbit animation. If time is short, this can be implemented as pure frontend state with no backend call at all (see 7.3).

```
{ "threat_id": "THREAT-001", "plan_id": "PLAN-3", "approved": true }
```

4.7 Field-naming rules everyone must follow

- snake_case for all JSON fields, no exceptions.
- All timestamps are ISO 8601 UTC strings.
- All distances in km, all velocities in km/s, all delta-v in m/s (note the unit switch — it's intentional and standard in this field, keep it consistent).
- risk_score and any other 0–1 score is always a float between 0.0 and 1.0, never a percentage string.
- IDs are always strings, even if they look numeric.

- status is always one of the enum strings shown in the examples above — never a lowercase or synonym variant.

5. Backend Spec — Member 2

5.1 Responsibilities

- FastAPI app exposing exactly the endpoints in Section 4.
- SGP4-based (or simplified two-body Keplerian) propagation of every object over a lookahead window (e.g. next 6 hours, 30s steps).
- Conjunction detection: for every object pair, find minimum separation distance within the window; flag pairs under a threshold (e.g. 5 km) as threats.
- Maneuver simulation: apply a delta-v to the primary object's orbital elements, re-propagate, re-run conjunction detection against every other object.
- Serve mock_scenario.json as the initial object set; support reset endpoint so the same demo scenario can be re-run live on stage.

5.2 Suggested module layout

```
backend/  
  main.py           # FastAPI app, route definitions only  
  models.py         # Pydantic models matching Section 4 exactly  
  propagation.py    # SGP4 / Keplerian propagation functions  
  conjunction.py    # pairwise distance search, risk scoring  
  maneuver.py       # apply delta-v, produce new orbital elements  
  simulator.py      # orchestrates: candidate -> propagate -> re-check all  
  data/mock_scenario.json  
  requirements.txt  # fastapi, uvicorn, sgp4, numpy, scipy, pydantic
```

5.3 Simplified risk score (do not over-engineer this)

A full collision-probability model (covariance-based P_c) is not worth the time in 24 hours. Use a defensible simplified heuristic and say so plainly if asked:

```
risk_score = clamp(1 - (miss_distance_km / SAFE_DISTANCE_KM), 0, 1) * velocity_factor  
# SAFE_DISTANCE_KM = 5.0 (tune for demo drama, keep physically plausible)  
# velocity_factor = min(relative_velocity_km_s / 10, 1.0) # higher closing speed -> higher urgency
```

5.4 Maneuver → new orbit (keep it simple and correct)

- raise_orbit: increase semi_major_axis_km by a fixed step (e.g. +2 km), compute delta_v via vis-viva equation.
- lower_orbit: same, negative step.
- phase_shift: adjust mean_anomaly_deg forward/back with a small altitude change for a set delay window, then return.
- wait: no orbital change, delay_min > 0 — re-check the threat later in the propagation window (tests whether it self-resolves).

5.5 Endpoint checklist

Method	Path	Purpose
GET	/api/scenario	Return the current object set (for frontend initial render)
POST	/api/scenario/reset	Reload mock_scenario.json (for repeatable live demo runs)
GET	/api/threats	Run detection, return conjunctions (Section 4.2)
POST	/api/plan	Return candidate maneuvers for a threat (4.3)
POST	/api/simulate	Counterfactual check for one candidate (4.4)

Method	Path	Purpose
POST	/api/decide	Full pipeline incl. Verifier re-check, final decision (4.5)
POST	/api/approve	Optional — human sign-off flag (4.6); can be frontend-only instead
WS	/ws/live	Optional — stream propagated positions for live globe animation

5.6 Verifier — do not build a new module for this

The Verifier is the same function as the Conflict Checker (5.1), called once more on the plan the Commander selected, after scoring. If it comes back infeasible, call the Commander again excluding the failed plan and try the next-best candidate — exactly once. Do not write a separate verification module or a general retry loop; reusing simulator.py's existing function is both less code and less risk than writing something new the night before a demo.

6. AI Agent Spec — Member 4

6.1 Orchestrator shape

Build this as plain Python classes calling the backend's simulation functions directly, plus Google Gemini for reasoning text. Do not reach for a heavy agent framework (LangGraph/CrewAI/AutoGen) unless someone on the team already knows one cold — in 24 hours, framework debugging time is the enemy. A ~150-line orchestrator is plenty and is easier to demo and explain to judges.

```
class ThreatDetector:
    def run(self, scenario) -> list[Threat]: ...           # pure backend call, no LLM

class ManeuverPlanner:
    def propose(self, threat) -> list[Candidate]: ...      # backend generates physical candidate
s;                                                         # Gemini call optionally annotates t
hem (cuttable)

class ConflictChecker:
    def check(self, candidate) -> SimResult: ...          # pure backend call, no LLM

class MissionGuardian:
    def weight(self, threat, candidate, sim_result) -> float: ... # criticality lookup, determini
stic

class Commander:
    def decide(self, threat, candidates, sim_results, weights) -> Decision:
        # deterministic scoring (6.4) + one Gemini call to write the justification text
        ...

class Verifier:
    def verify(self, decision) -> VerifyResult:
        return conflict_checker.check(decision.selected_plan) # same function, called again – see
2.4

class Orchestrator:
    def resolve(self, threat_id):
        threat = ...
        candidates = planner.propose(threat)
        results = [checker.check(c) for c in candidates]
        weights = [guardian.weight(threat, c, r) for c, r in zip(candidates, results)]
        decision = commander.decide(threat, candidates, results, weights)
        verify_result = verifier.verify(decision)
        if not verify_result.feasible:
            decision = commander.decide(threat, exclude(candidates, decision.selected_plan), resul
ts, weights)
            verify_result = verifier.verify(decision) # exactly one replan attempt, then stop –
never loop further
        return decision, verify_result
```

6.2 Where Gemini calls go (exactly two, keep it tight — locked per 2.4)

Call	Input	Output used for
Planner annotation (optional, cut first if short on time)	Threat details + candidate physical results	One-sentence rationale per candidate shown in the decision-trace panel
Commander justification	Selected plan + rejected plans + reasons + criticality + verification result	The final justification paragraph shown on the dashboard and read aloud in the demo

Both calls are single-shot: Python has already run every tool (propagation, detection, simulation) before Gemini is ever called, and the full structured JSON result is handed to it in one prompt. This is deliberately not a live

multi-turn function-calling loop (2.4) — it reads as tool-augmented reasoning to judges without the mid-demo failure risk of a real request-tool / execute / return loop. Use Gemini's structured output mode (response schema) so you get back predictable JSON fields instead of parsing free text.

6.3 Sample prompt skeleton (Commander justification)

```
SYSTEM: You are the Commander agent in a satellite conjunction response system.
You are given already-computed, final structured results, including a completed
verification pass. Never invent, change, or override any number or safety status.
Write a 2-3 sentence plain-English justification for the selected plan, referencing
the rejected plans and why they were unsafe or costly. Return structured JSON
matching the provided response schema.
```

```
USER: {
  "selected_plan": "PLAN-3", "scores": {...}, "verification": {"status": "SAFE", ...},
  "rejected_plans": [{"plan_id": "PLAN-1", "reason": "..."}, ...]
}
```

6.4 Commander scoring formula (deterministic, in Python — never Gemini)

```
total_score = (
    0.5 * safety_score          # 1.0 if primary resolved and 0 secondary conflicts, scaled down
    otherwise
    + 0.2 * (1 - normalized_delta_v)
    + 0.2 * mission_criticality_weight # critical=1.0, standard=0.6, low=0.3
    + 0.1 * (1 - normalized_delay)
)
# Only plans with feasible == true are eligible. Highest total_score wins.
# Tie-break: prefer lower delta_v, then lower delay.
```

6.5 Memory / state across the pipeline

The orchestrator keeps the full trace in memory for one threat resolution: threat → all candidates tried → all simulation results → all rejection reasons → final decision → verification result. This trace is exactly what the frontend's decision-trace panel renders, and it is the concrete evidence of 'agentic reasoning' for judges — show them the trace object, not just the final answer.

7. Frontend & Dashboard Spec — Member 3

7.1 Layout — 3 fixed panels

- **Left — Threat monitor:** satellite, debris, closest approach, time to encounter, risk level, relative velocity for each active threat. Click a threat to drive the other two panels.
- **Center — Orbit visualization:** Earth with the two objects' orbits, camera following the conjunction point; animates before/after once the maneuver is approved (orbit line shifts).
- **Right — Agent decision trace:** live checklist tagged by agent: [THREAT DETECTOR] detected → [MANEUVER PLANNER] generated 3 candidates → [CONFLICT CHECKER] Plan 1 simulated / secondary conflict found → [MISSION GUARDIAN] Plan 1 rejected → [COMMANDER] Plan 3 selected → [VERIFIER] final risk LOW → awaiting approval.

7.2 The "why was this rejected?" card — one of the two strongest visual elements

Every plan, rejected or selected, renders as a card with the same fixed fields so judges can visually compare them at a glance. Pull every field directly from the `/api/simulate` and `/api/decide` responses (Section 4.4–4.5) — do not paraphrase or reformat them client-side.

Field	PLAN-1 example	PLAN-3 example
Status	REJECTED	SELECTED
Primary threat	Resolved	Resolved
Secondary consequence	Creates conjunction with SAT-05	None
Closest approach	1.4 km	15.4 km
Reason	Although Plan 1 resolves the original debris threat, its simulated trajectory creates a new high-risk conjunction with SAT-05.	—

7.3 Human approval control

Once the Verifier returns SAFE, show an "Approve maneuver" button. Clicking it can purely flip frontend state to trigger the before/after orbit animation — it does not need to call the backend at all if time is short (Section 4.6 is optional). Label the button's context clearly: this approves a simulated recommendation, it does not fire real thrusters.

7.4 Tech choices

Piece	Choice	Why
Framework	React + Vite	Fast dev server, minimal config, standard
3D globe	CesiumJS (primary) or Three.js (fallback, see 9.2)	Cesium looks the most 'real aerospace'; Three.js is faster to get working if Cesium setup stalls past hour 3
Styling	Tailwind CSS + a small custom HUD theme (7.4)	Fast to build, easy to make look non-generic with a few custom rules
Data	REST polling every 2–3s to start; upgrade to WebSocket only if time allows	Polling is trivial and demo-safe; don't risk WS debugging late at night

Piece	Choice	Why
Charts/telemetry	Plain SVG or a minimal library (recharts) for delta-v/risk bars	Keep dependencies light

7.5 State flow

- On load: GET /api/scenario, render all objects on the globe.
- Poll GET /api/threats; when a HIGH severity threat appears, auto-select it and highlight both objects.
- On selection: POST /api/decide for that threat_id (backend runs the whole pipeline server-side and returns the full trace) — simplest possible integration, only one call needed from the frontend per resolution.
- Render the response's rejected_plans, selected_plan, and verification into the decision-trace panel in order, with a short stagger animation (300–500ms per step) so the pipeline reads like it's happening live, even though the backend already finished computing it. Show the Approve button only after verification.status is SAFE.

7.6 Make it look like a real aerospace console, not an 'AI demo'

- Dark background (near-black navy, not pure black), thin 1px borders, monospace font (e.g. 'JetBrains Mono' or 'IBM Plex Mono') for all numeric telemetry.
- No rounded bubbly cards, no purple-to-pink AI-startup gradients, no emoji in the UI chrome (keep the ■ style severity glyphs minimal and monochrome-accented, not colorful).
- Reference aesthetic: SpaceX mission control overlays, NASA JPL 'Eyes on the Solar System', ESA's Space Debris Office dashboards — precise data readouts, subtle grid lines, restrained color (one accent cyan/amber, not a rainbow).
- Use real units and precision everywhere (e.g. ' Δv 2.8 m/s', 'Miss dist. 0.42 km', 'TCA 14:32:10 UTC') — specificity reads as credible, vague labels read as a mockup.
- Animate the rejection: when a plan fails, briefly flash the new conflicting object in the 3D view before the panel marks the plan ✗ — this single beat is what makes the 'AI checks itself' story land visually.

8. Integration, Git Workflow & Demo Script — Member 1

8.1 Repository structure

```
orbitguard/
  backend/          # Member 2
  agent/            # Member 4
  frontend/         # Member 3
  contracts/        # copies of the Section 4 schemas, kept in sync, source of truth
  data/mock_scenario.json
  README.md         # setup + run instructions, judges may read this
  docs/OrbitGuard_Project_Bible.pdf # this document
```

8.2 Branching

- main stays runnable at all times.
- One branch per member: feat/backend, feat/agent, feat/frontend.
- Merge to main at every timeline checkpoint (Section 3), not just at the end — this satisfies the 'commit every 3–6 hours' rule and de-risks a broken final merge.
- Whoever changes a Section 4 contract field updates contracts/ and posts it to the group chat before writing code against it.

8.3 Integration checkpoints (do not skip these)

Checkpoint	What must work end-to-end
Hour 4	Frontend renders mock_scenario.json objects; agent orchestrator runs against a hardcoded threat
Hour 8	Frontend calls real GET /api/threats and lists real threats
Hour 12	Full loop: frontend triggers POST /api/decide, agent's Gemini calls run, response (incl. verification) renders in the trace panel
Hour 16	Full demo run works twice in a row without a restart
Hour 20	Demo run is rehearsed on the exact machine that will present

8.4 Demo script (2–3 min video / live pitch)

- 0:00–0:15 — The one-sentence pitch (Section 1.2). State clearly it's a decision-support prototype.
- 0:15–0:40 — Show the alert firing: SAT-03 / DEB-17, risk score, miss distance, TCA.
- 0:40–1:30 — The core moment: Plan 1 tried, rejected on screen because it creates a new conflict with SAT-05, then Plan 2 also rejected. Narrate this explicitly: 'the system just discovered a problem with its own idea and is retrying — twice.'
- 1:30–2:00 — Plan 3 selected and verified SAFE, show the Gemini justification text and click Approve as the maneuver animates on the globe.
- 2:00–2:30 — One sentence each on the agent pipeline and why it's reliable (deterministic physics + Gemini reasoning only, per Section 2.3), plus the line from 1.3: decision-support prototype, not real spacecraft control.
- 2:30–3:00 — Real-world framing: this is what an operator dashboard for conjunction response could look like, with a human still approving the final call.

9. Fallback Plans — Read Before You Panic

9.1 If SGP4 propagation is taking too long

Fall back to simplified two-body Keplerian propagation (no perturbations). It's less accurate but is entirely sufficient for a controlled synthetic scenario and takes a fraction of the implementation time. Nobody will penalize you for this if the demo works.

9.2 If CesiumJS setup stalls past hour 3

Switch immediately to a Three.js textured sphere with simple orbit lines (ellipses) drawn as geometry — visually still reads as '3D space view' to judges and is much faster to get working. Do not lose more than 30 minutes deciding; just switch.

9.3 If the Gemini API calls are too slow or flaky mid-demo

Pre-generate and cache the justification text for the exact demo scenario ahead of time as a fallback string keyed by `threat_id/plan_id`, and use it if the live call doesn't return within ~3 seconds. The deterministic scoring and verification still run live — only the narrated text has a safety net.

9.4 If integration breaks near the end

Fall back to the last known-good checkpoint branch (Section 8.3) and present that. A working demo of hour-16 functionality beats a broken demo of hour-22 functionality. Never present live off a branch that hasn't been tested end-to-end at least once.

9.5 If time is genuinely short, cut in this order

- 1st cut: WebSocket live streaming — REST polling is fine.
- 2nd cut: `phase_shift` maneuver type — 3 candidate types (raise/lower/wait) still tell the full story.
- 3rd cut: Planner's Gemini annotation call — keep only the Commander's justification call, it's the one judges hear.
- 4th cut: real `/api/approve` backend call — make Approve a pure frontend state flip (Section 4.6, 7.3).
- Never cut: the counterfactual rejection moment (Section 8.4, 0:40–1:30), the Verifier re-check, or the deterministic-physics-only rule (Section 2.3–2.4) — those three are the entire pitch and its credibility.

10. Final Scope Lock — Read This Last, Then Stop Reading Feature Ideas

The finished prototype MUST demonstrate all 14 of the following. If they all work, stop adding features and switch fully to reliability, visual clarity, README, and demo rehearsal.

- One protected satellite, several orbital objects, one dangerous conjunction.
- Threat detection with a real risk score.
- 3–4 candidate maneuvers generated.
- Deterministic maneuver simulation.
- Secondary conflict detection.
- At least one rejected maneuver, with reason shown on screen.
- Gemini-powered reasoning/explanation.
- Deterministic final selection (Commander scoring formula).
- Final verification pass (Verifier).
- Human approval step before the animation plays.
- Visible agent decision trace, tagged by agent name.
- Before/after orbital visualization.
- Clear on-screen labeling as a decision-support prototype, controlled scenario.
- Public GitHub repo with regular commits + a 2–3 minute demo video, per the hackathon submission rules.

10.1 Explicitly out of scope — do not add these under any circumstances

- Machine-learning collision prediction or model training.
- Real satellite control or real thruster commands.
- Hundreds of satellites or full global space traffic management.
- Mobile app, authentication, a complex database, blockchain, IoT hardware.
- Unnecessary WebSockets, multiple AI providers, or five separate LLM API pipelines.
- Complicated memory systems or full probability-of-collision mathematics.

10.2 The one sentence that is the whole project

"What if the maneuver that saves a satellite from one collision creates another collision? OrbitGuard answers this by simulating the consequences of its own proposed actions before accepting them."
Everything in this document exists to make that sentence demonstrably true on stage.